

Amirkabir University of Technology
(Tehran Polytechnic)

Process-level task-offloading between two computers using process migration mechanism

By Mohammad Ali Moghimi

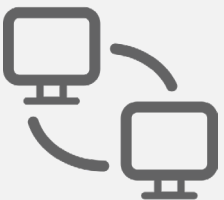
Supervisor: Dr. Hamid Reza Zarandi

July 2024



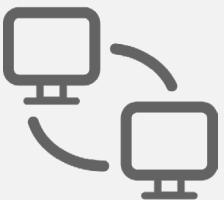
Outline

- Introduction
- Related Work
- Process-level task-offloading requirements
- Architecture and Implementation
- Evaluation and Results
- Conclusion and Future Work



Introduction

- Task-Offloading → a solution
- Applications:
 - Autonomous Driving
 - Surveillance
 - Load Balancing
- **Challenge: how to separate computation-intensive tasks from the others?**



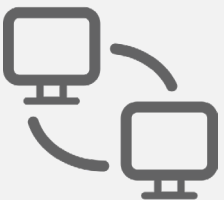
Introduction (cont.)

- Proposed approach:

Separation and offloading at the process level

- Advantages:
 - + Elimination of direct interaction between developer and cloud
 - + Automatic identification of computation-intensive task, based on OS statistical information

└─→ isn't covered in this work



Related Work

Thread-State Migration

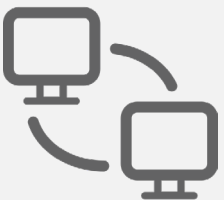
- Migrates thread state from one ALVM to another
- Limitations:
 - Requires massive modifications of ALVMs
 - The modified ALVMs severely locally running applications

Code Migration

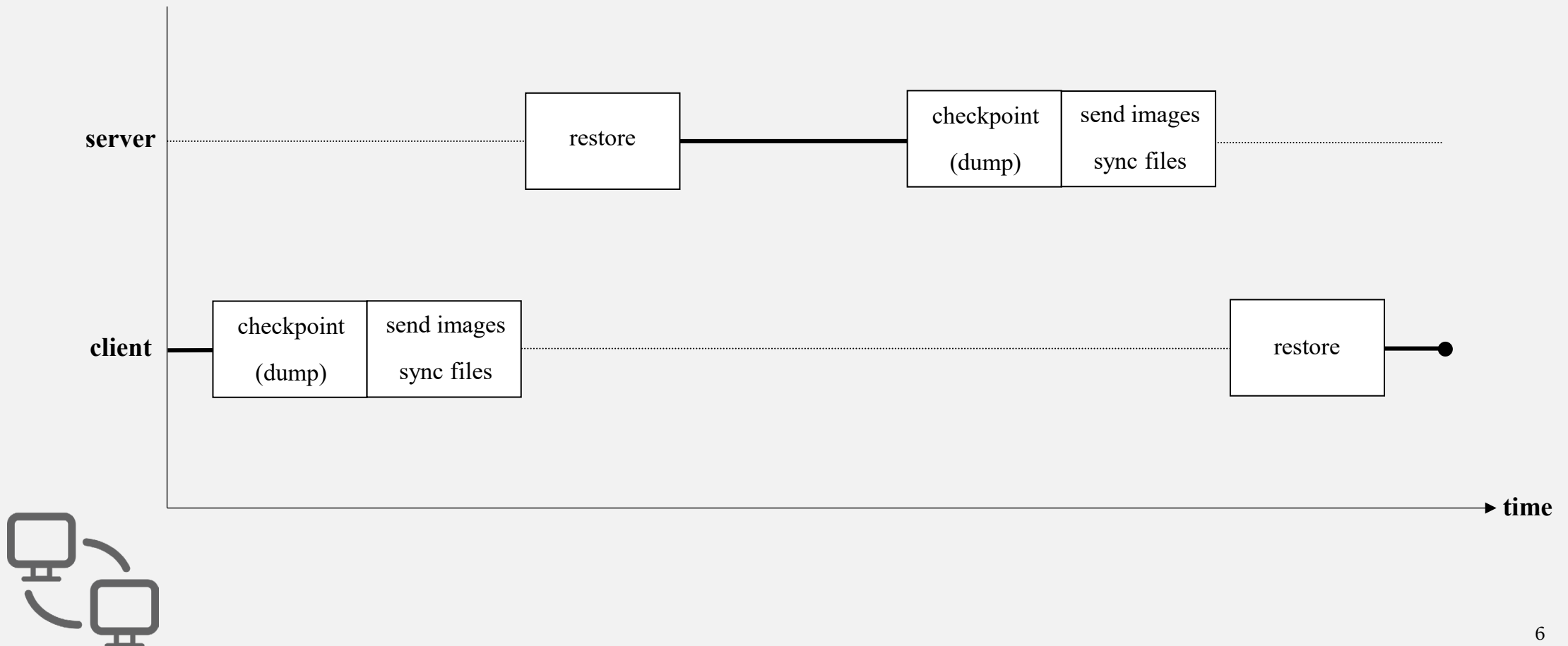
- The most common technique
- Two approaches:
 - 1) Service-oriented client/server setup
 - 2) Platform-independent code transfer
- Limitations:
 - Requires application source code modification
 - Source code must be available at the cloud

VM Clone Migration

- Transforms a synchronized mirror of the OS to the cloud
- Limitations:
 - Adds too much communication overhead
 - Results in inefficient use of resources



Process-level task-offloading requirements



Process-level task-offloading requirements (cont.)

Checkpoint/restore tool

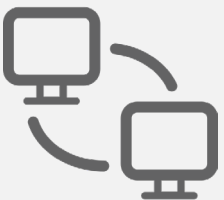
- Checkpoints a process in the client
- Restores the process on the server

File syncing tool

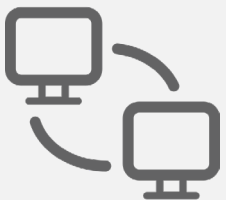
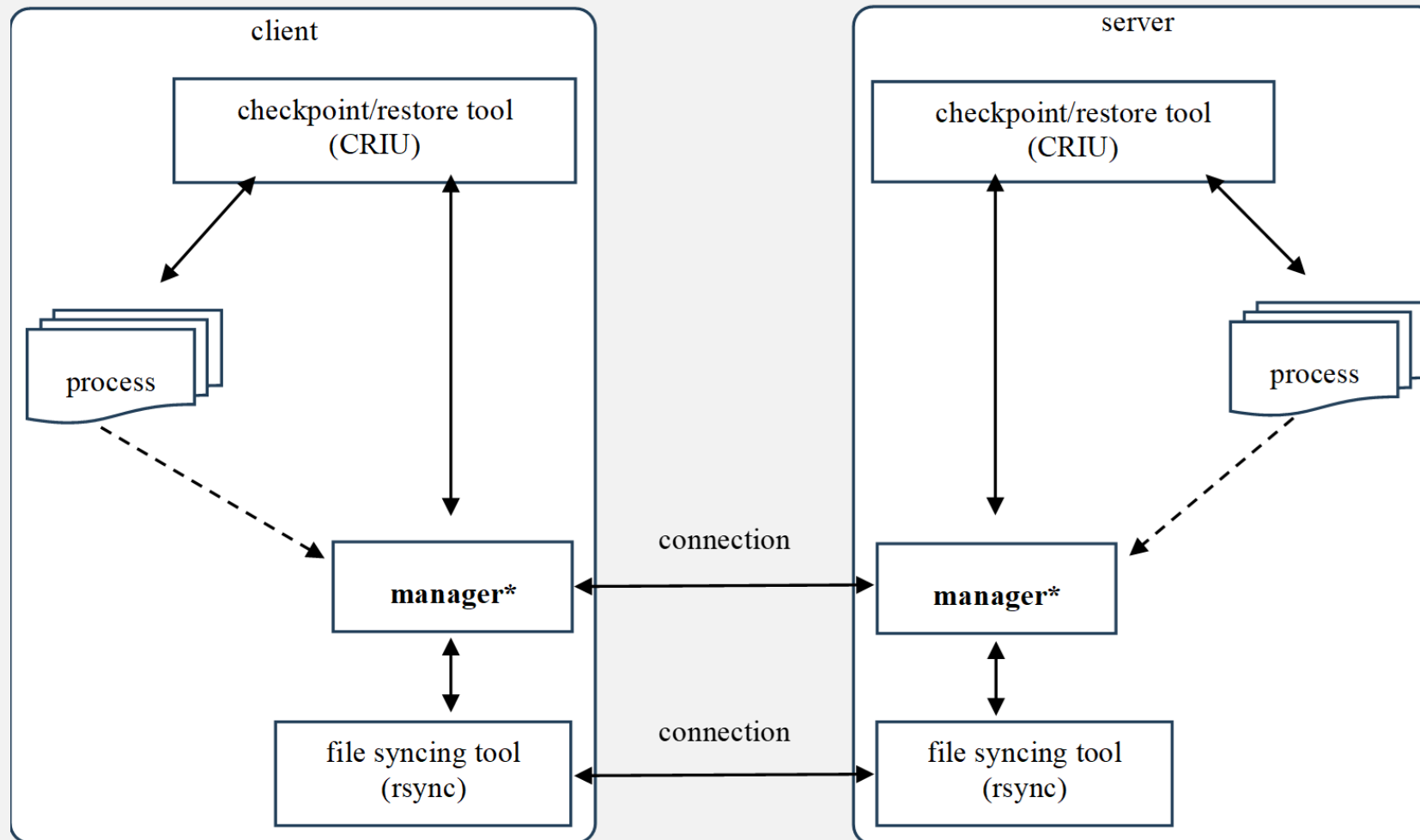
- Sends the checkpoint from the client to the server
- Synchronizes files that the process depends on for execution

Manager

- Coordinates the workflow



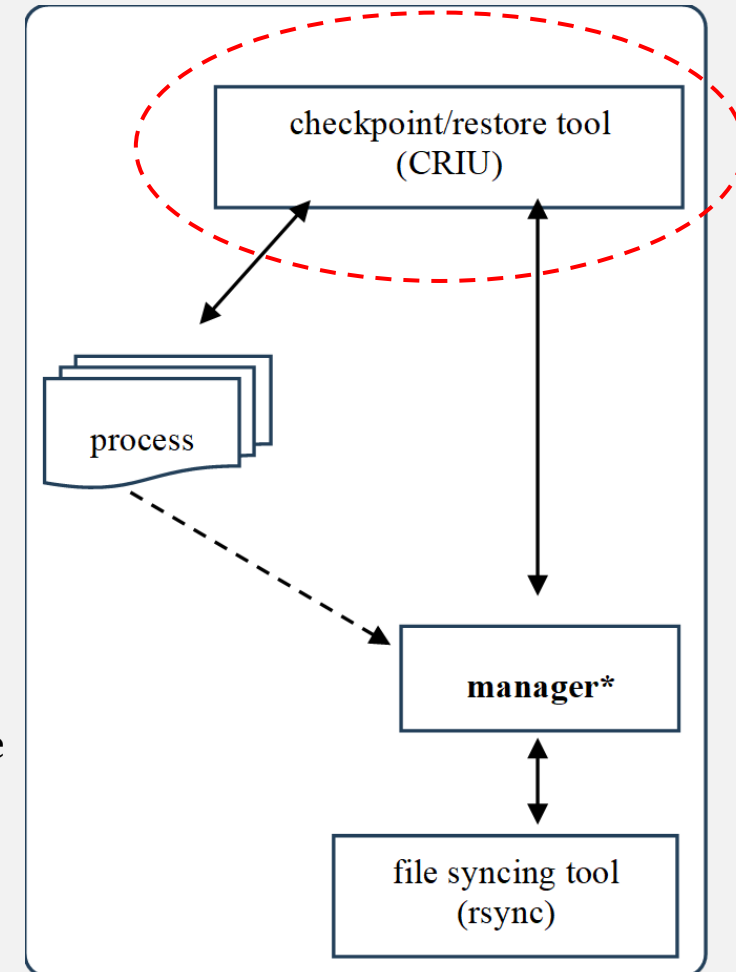
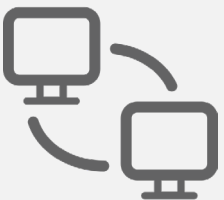
Architecture and Implementation



Architecture and Implementation (cont.)

Checkpoint/restore tool: CRIU

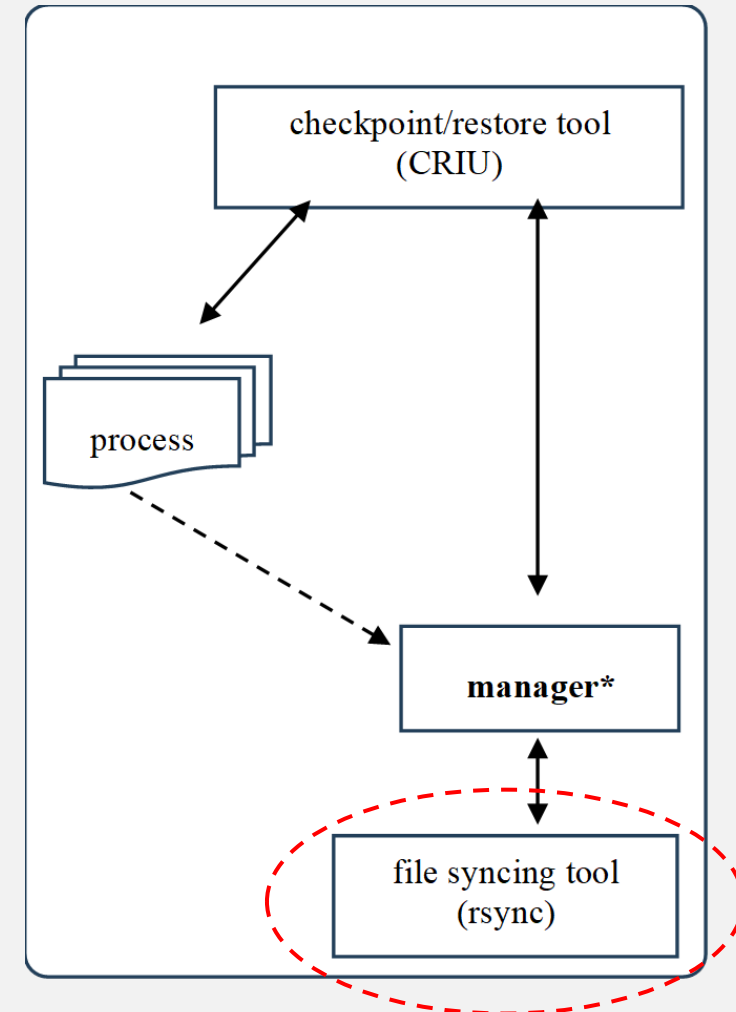
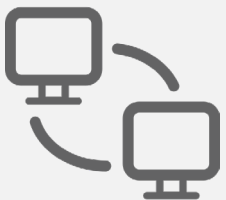
- Stores information such as memory maps and file descriptors into image files
- Restores a process using the image files
- Advantages:
 - + Runs in user space
 - + Works in in a wider range of scenarios
- Limitations:
 - The checkpointed process PID must be free in the server
 - Libraries and the files that the process depends on must be the same in client and server



Architecture and Implementation (cont.)

File syncing tool: rsync

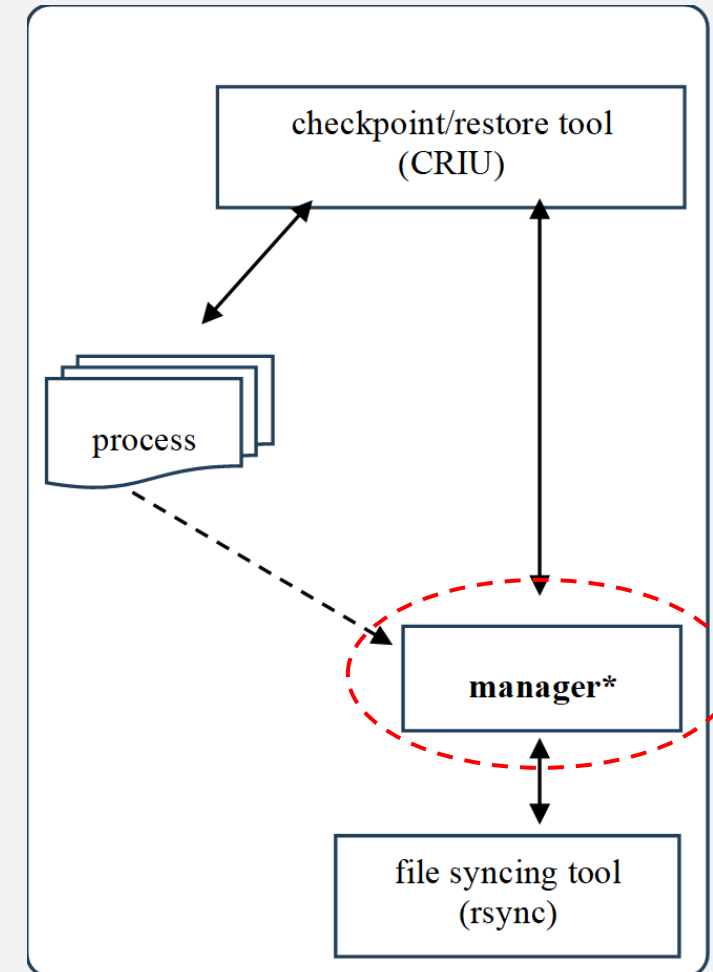
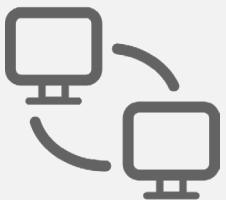
- Used to synchronize:
 - 1) Files the process depends on for execution
 - 2) Image files resulted from checkpointing the process
- Sends the data over SSH
- Only sends part of files that are not identical
- Compresses the data before sending



Architecture and Implementation (cont.)

Manager

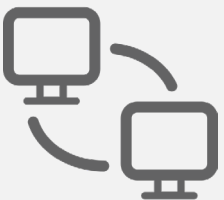
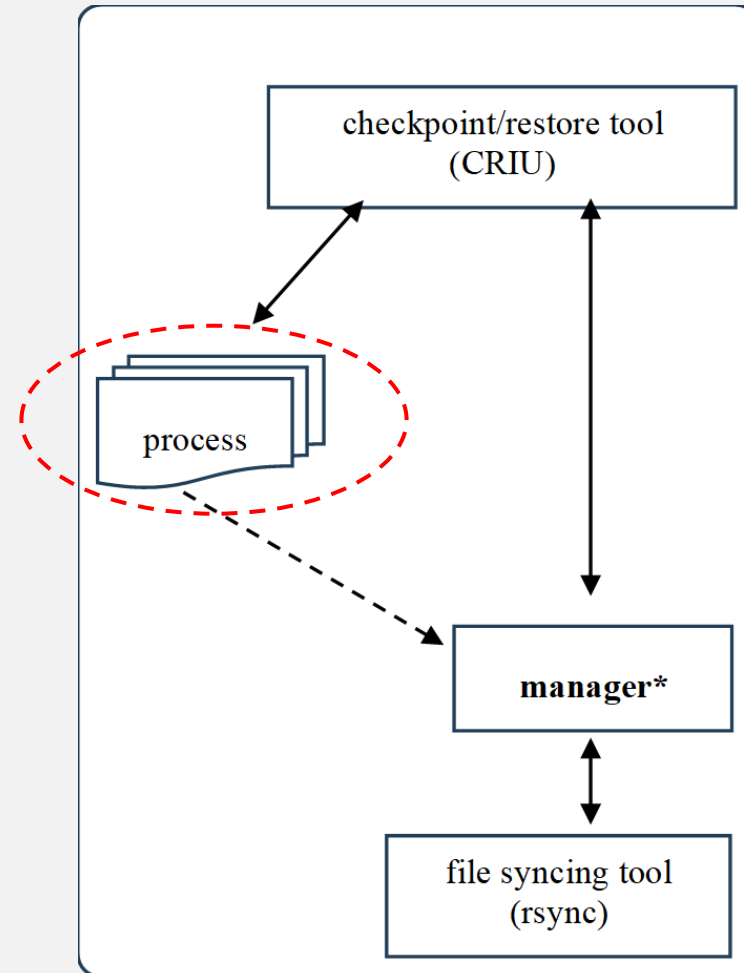
- Two TCP servers corresponding to two flows at manager:
 - 1) communicates with local processes
 - a) Receives offloading request from a local process
 - b) Coordinates the tools to checkpoint the process and synchronize the files
 - c) Sends notification to the peer
 - 2) communicates with peer's manager
 - a) Receives notification from the peer
 - b) Restore the process



Architecture and Implementation (cont.)

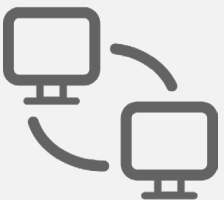
Process

- Must send offloading request to the manager twice
- Remarks:
 - The first request must be sent when all the files required by the process are opened
 - The second request must be sent before the process terminates



Evaluation and Results

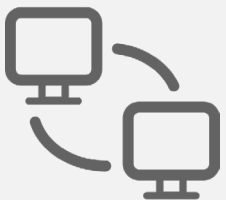
- Used four tests from MiBench and Splash-3 benchmarks
- Measured runtime with and without offloading
- Evaluated under different server specification



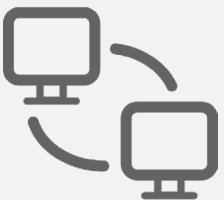
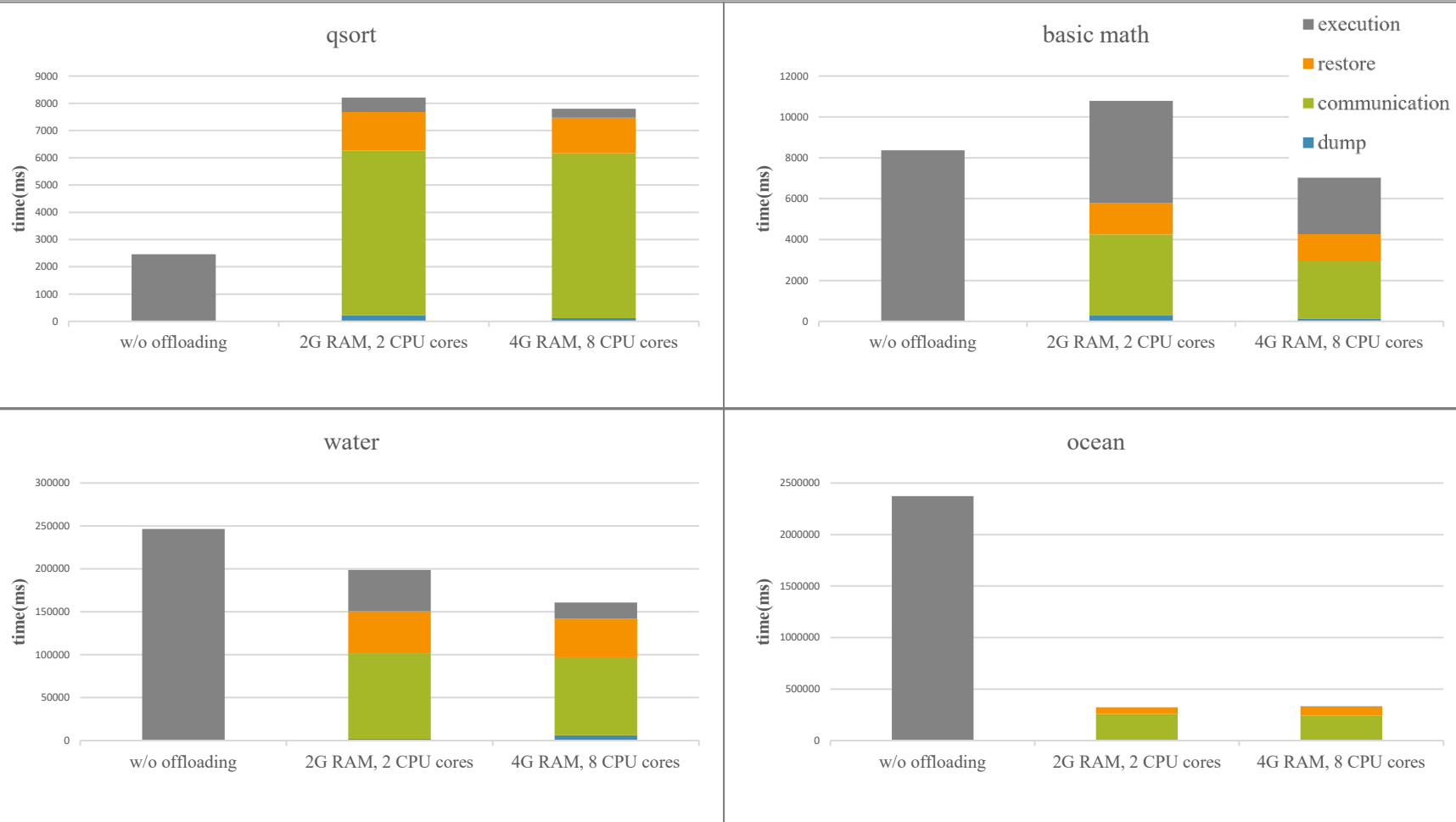
Evaluation and Results (cont.)

Runtime of test programs

client spec	test	without offloading	with offloading								
		total time (ms)	server spec	total time (ms)	dump (ms) (client)	send (ms) (client to server)	restore (ms) (server)	execution (ms) (server)	dump (ms) (server)	send (ms) (server to client)	restore (ms) (client)
1G RAM; 1 CPU core	basic math	8367	2G RAM; 2 CPU cores	10790	157	1298	536	5013	153	2642	993
			4G RAM; 8 CPU cores	7032	29	1240	573	2763	91	1579	755
	qsort	2458	2G RAM; 2 CPU cores	8204	82	2343	603	527	143	3697	815
			4G RAM; 8 CPU cores	7802	50	3373	529	332	73	2670	776
	water	246318 (~ 4 minutes)	2G RAM; 2 CPU cores	198964 (~ 3 minutes)	604	6650	692	47941	1348	92953	48543
			4G RAM; 8 CPU cores	160923 (~ 2 minutes)	533	4029	480	18757	5471	86996	44354
	ocean	2371780 (~ 39 minutes)	2G RAM; 2 CPU cores	323091 (~ 5 minutes)	649	4608	786	3945	1886	253184	58093
			4G RAM; 8 CPU cores	332892 (~ 5 minutes)	591	3732	529	2708	5913	232208	86478

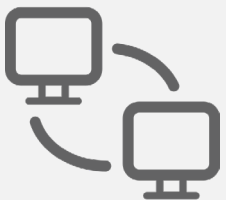
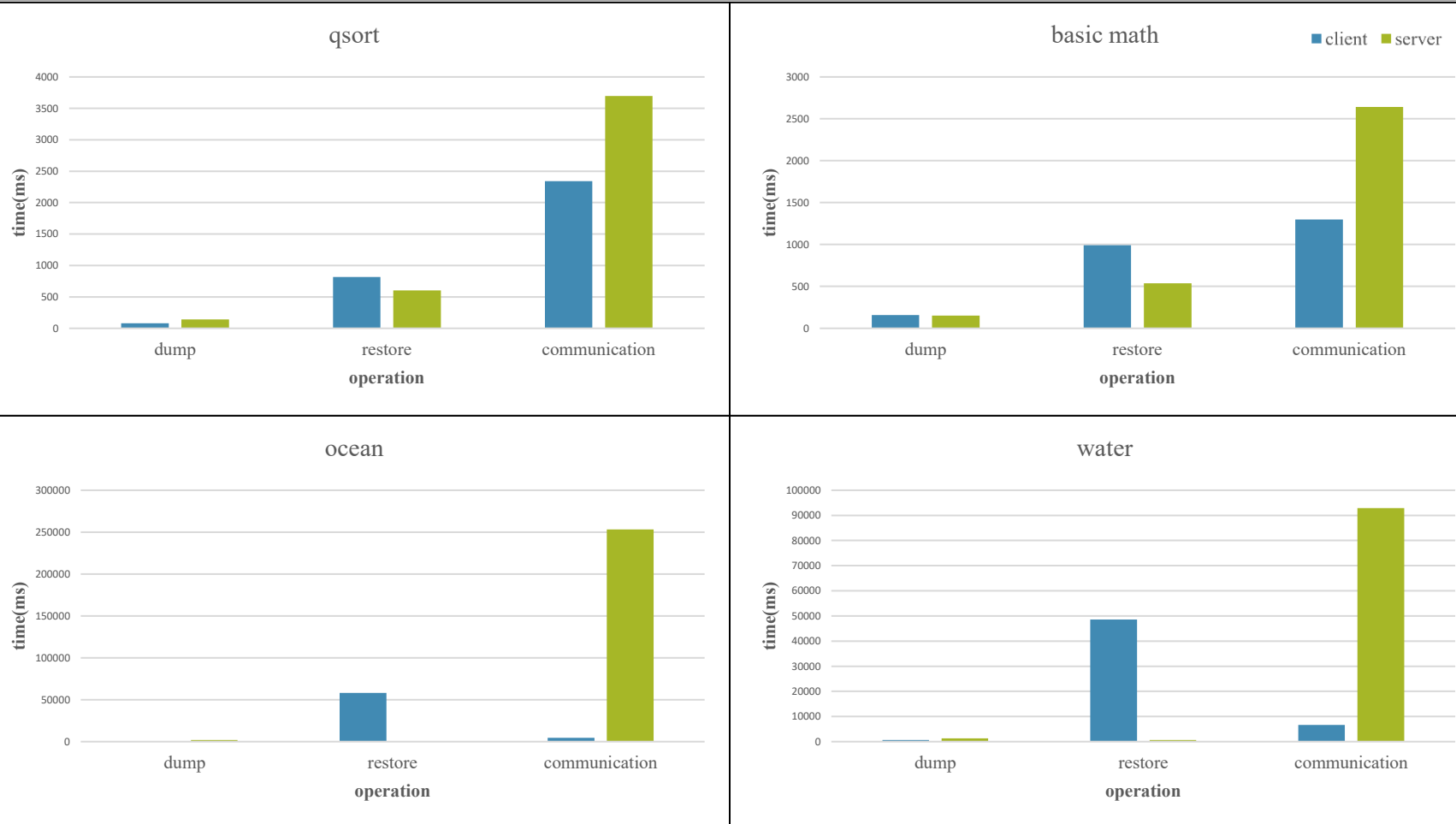


Evaluation and Results (cont.)



Contribution of different operations to the total runtime

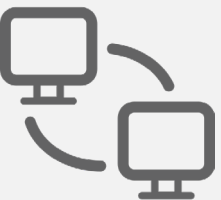
Evaluation and Results (cont.)



Comparison of the runtime of different operations on the client and the server

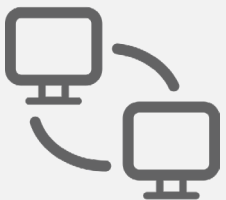
Conclusion and Future Work

- My process-level task offloading system:
 - Efficient for computation-intensive tasks
 - Dependent on the checkpoint/restore tool
 - High overhead when the process returns to the client
- Future Work:
 - Automatic identification of computation-intensive task, based on OS statistical information
 - Optimization of the process return mechanism to reduce overhead



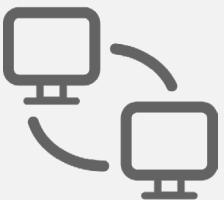
References

- [1] J. Wang, J. Pan, F. Esposito, P. Calyam, Z. Yang, and P. Mohapatra, "Edge cloud offloading algorithms: Issues, methods, and perspectives," *ACM Computing Surveys (CSUR)*, vol. 52, no. 1, pp. 1-23, 2019.
- [2] A. Yousafzai, I. Yaqoob, M. Imran, A. Gani, and R. M. Noor, "Process migration-based computational offloading framework for IoT-supported mobile edge/cloud computing," *IEEE internet of things journal*, vol. 7, no. 5, pp. 4171-4182, 2019.
- [3] B. Wang, C. Wang, W. Huang, Y. Song, and X. Qin, "A survey and taxonomy on task offloading for edge-cloud computing," *IEEE Access*, vol. 8, pp. 186080-186101, 2020.
- [4] M. Cui, S. Zhong, B. Li, X. Chen, and K. Huang, "Offloading autonomous driving services via edge computing," *IEEE Internet of Things Journal*, vol. 7, no. 10, pp. 10535-10547, 2020.
- [5] Z. Xiao, J. Shu, H. Jiang, G. Min, H. Chen, and Z. Han, "Perception task offloading with collaborative computation for autonomous driving," *IEEE Journal on Selected Areas in Communications*, vol. 41, no. 2, pp. 457-473, 2022.
- [6] X. Chen, and G. Liu, "Energy-efficient task offloading and resource allocation via deep reinforcement learning for augmented reality in mobile edge networks," *IEEE Internet of Things Journal*, vol. 8, no. 13, pp. 10843-10856, 2021.
- [7] S. Imai, and C. A. Varela, "Light-weight adaptive task offloading from smartphones to nearby computational resources." pp. 146-152.
- [8] P. Chen, L. Luo, D. Guo, X. Luo, X. Li, and Y. Sun, "Secure Task Offloading for Rural Area Surveillance Based on UAV-UGV Collaborations," *IEEE Transactions on Vehicular Technology*, 2023.
- [9] M. T. Chung, J. Weidendorfer, K. Förlinger, and D. Kranzlmüller, "Proactive task offloading for load balancing in iterative applications." pp. 263-275.
- [10] A. Tošić, "Run-time application migration using checkpoint/restore in userspace," *arXiv preprint arXiv:2307.12113*, 2023.
- [11] A. Yousafzai, A. Gani, R. M. Noor, A. Naveed, R. W. Ahmad, and V. Chang, "Computational offloading mechanism for native and android runtime based mobile applications," *Journal of Systems and Software*, vol. 121, pp. 28-39, 2016.



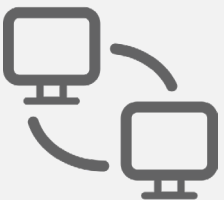
References (cont.)

- [12] D. Kovachev, T. Yu, and R. Klamma, "Adaptive computation offloading from mobile devices into the cloud." pp. 784-791.
- [13] B.-D. Lee, "A framework for seamless execution of mobile applications in the cloud," *Recent Advances in Computer Science and Information Engineering: Volume 3*, pp. 145-153: Springer, 2012.
- [14] R. K. Ma, and C.-L. Wang, "Lightweight application-level task migration for mobile cloud computing." pp. 550-557.
- [15] T. Verbelen, T. Stevens, P. Simoens, F. De Turck, and B. Dhoedt, "Dynamic deployment and quality adaptation for mobile augmented reality applications," *Journal of Systems and Software*, vol. 84, no. 11, pp. 1871-1882, 2011.
- [16] E. Y. Chen, and M. Itoh, "Virtual smartphone over IP." pp. 1-6.
- [17] G. Portokalidis, P. Homburg, K. Anagnostakis, and H. Bos, "Paranoid android: versatile protection for smartphones." pp. 347-356.
- [18] M. S. Gordon, D. A. Jamshidi, S. Mahlke, Z. M. Mao, and X. Chen, "{COMET}: Code offload by migrating execution transparently." pp. 93-106.
- [19] H. A. Alameddine, S. Sharafeddine, S. Sebbah, S. Ayoubi, and C. Assi, "Dynamic task offloading and scheduling for low-latency IoT services in multi-access edge computing," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 3, pp. 668-682, 2019.
- [20] C. Press, "IoT Fundamentals: Networking Technologies, Protocols, and Use Cases for the Internet of Things," 2017.
- [21] "CRIU official website," <https://criu.org/>
- [22] "DMTCP official website," <https://dmtcp.sourceforge.io/>
- [23] J. Duell, "The design and implementation of berkeley lab's linux checkpoint/restart," 2005.
- [24] "OpenVZ official website," <https://openvz.org/>
- [25] "SCP documentation," <https://www.man7.org/linux/man-pages/man1/scp.1.html>.



References (cont.)

- [26] "Syncthing documentation," <https://docs.syncthing.net/>
- [27] "FreeFileSync official website," <https://freefilesync.org/>
- [28] "Rclone official website," <https://rclone.org/>
- [29] "rsync official website," <https://rsync.samba.org/>
- [30] Y .Chen, "Checkpoint and restore of micro-service in docker containers." pp. 915-918.
- [31] A. Tridgell, and P. Mackerras, "The rsync algorithm," 1996.
- [32] M. R. Guthaus, J. S. Ringenber, D. Ernst, T. M. Austin, T. Mudge, and R. B. Brown, "MiBench: A free ,commercially representative embedded benchmark suite." pp. 3-14.
- [33] C. Sakalis, C. Leonardsson, S. Kaxiras, and A. Ros, "Splash-3: A properly synchronized benchmark suite for contemporary research." pp. 101-111.
- [34] J. P. Singh, W.-D. Weber, and A. Gupta, "SPLASH: Stanford parallel applications for shared-memory," *ACM SIGARCH Computer Architecture News*, vol. 20, no. 1, pp. 5-44, 1992.



Thank you for your attention

